SHIR — User Guide

How to use SHIR: what it does, how your data must be named, the step-by-step workflow, every reconstruction it can run, and the outputs it produces.

- **Installing / launching the standalone app:** Windows → `demands.md`, macOS → `demands_mac.md`.
- **Building the app from source:** `compilation_instruction.md`.

---

1. What the app does

SHIR loads **Siemens TWIX** `.dat` raw k-space files and reconstructs images with several pipelines, displays them, and exports PDF reports. It handles four acquisition types and a retrospective-undersampling simulator:

Acquisition	What it is	Reconstructions available
Fully-Sampled	Cartesian, fully sampled	RSS, or ESPIRiT + SENSE
Equispaced	Cartesian, uniformly undersampled + ACS	GRAPPA, GRAPPA+SENSE, SENSE, CS-SENSE, ESPIRiT (+L1/L2/TV)
Band	Non-Cartesian rotated "bands" (0–15)	NUFFT gridding → ESPIRiT + SENSE
Poisson-Disc	Non-Cartesian 2-D Poisson-disc	Zero-fill (RSS), or CS PICS (L1-wavelet)
Unknown	Any scan whose type isn't recognised from the filename	Generic fallback — Cartesian IFFT + RSS, NUFFT + RSS (trajectory JSON or assumed-radial), or CS (mask from k-space)
Simulator	Retrospective undersampling of a fully-sampled scan	None / GRAPPA / gGRAPPA / SENSE / CS-SENSE / ESPIRiT (+L1/L2/TV)

Every reconstruction can run on one of two engines — **BART** (exact) or the built-in **sigpy** (no external install) — see §6.

---

2. Data naming format (read this first)

The app **auto-detects each `.dat` file's type from its filename**, and finds the matching trajectory/mask JSON by folder layout. Getting the naming right is what makes files show up correctly.

Expected scan-folder layout

```
SE_1mm/                                ← a "scan folder" (any name)
├── dat_files/                         ← the .dat files live here
│   ├── band_se_0.dat … band_se_15.dat   (or band_tse_0.dat … for TSE)
│   ├── fully_sampled.dat
│   ├── equispaced_R4_acs24.dat
│   └── PoissonDisc.dat
├── Bands.json                         ← band trajectories (keys "0"–"15")
├── PoissonDisc.json                   ← Poisson-disc trajectory
└── Undersampled4equispaced.json       ← equispaced PE mask (the "4" = R factor)
```

You can open either the scan folder (`SE_1mm/`) or the `dat_files/` folder directly.

Filename → type rules (case-insensitive substring of the filename)

Type	Filename must contain	Extra rules	Trajectory JSON (in scan-folder root)
Band	`band_se_N` or `band_tse_N`	`N` = band index 0–15	`Bands.json`
Fully-Sampled	`fully_sampled`	—	*(none needed)*
Equispaced	`equispaced`	R factor read from `_R<n>` (e.g. `_R4`); ACS from the name	`Undersampled<R>equispaced.json`
Poisson-Disc	`poisson`	—	`PoissonDisc.json`

Anything not matching these is listed as **Unknown** — but it is **still reconstructed** with a generic fallback (see §5), so you always get an image. To rebuild an unknown **non-Cartesian** scan with NUFFT, drop a trajectory JSON next to the `.dat` named either `<scanname>.json` (same stem as the `.dat`) or `Trajectory.json` — same `{index: [[kx…],[ky…]]}` format as `Bands.json` . You can also right-click the file in the tree and pick a JSON manually. Tip: keep the standard JSON files in the **scan-folder root** (the parent of `dat_files/`), not inside `dat_files/` .

> **TSE scans:** files named `band_tse_N` are handled as Band data. For the Cartesian recons the app auto-detects TSE from the filename and un-scrambles the echo order; you can override this in the **TSE echo order** panel.

Siemens scanner scans (native product sequences)

Scans acquired with the scanner's **own** product sequences (spin-echo `t1_se_tra` , GRAPPA-accelerated gradient-echo `t1_fl2d_tra` , …) instead of the custom Pulseq sequences follow the same type keywords. Two things are worth understanding — what the filename controls, and what the app figures out from the data itself.

What the filename controls:

- **The type keyword decides the pipeline:** the name must contain `fully_sampled` (→ Fully-Sampled) or `equispaced` (→ Equispaced) — the raw scanner export (`… _t1_fl2d_tra_with_4.dat`) contains neither, so **rename it** to add the keyword (and `_R<n>_acs<n>`), or right-click → **Reconstruct as**.
- **The `Siemens_` prefix** (a filename starting with `Siemens`) tells the **Equispaced** recon to (a) **ignore any custom trajectory JSON** — the product sequence didn't use one — and (b) always **prompt for the PE-mask method** (analytical vs. `bart pattern`). It also biases reference matching toward the spin-echo scan (see §4c). It is otherwise optional.

What the app detects from the data (not from the name):

- **Multi-slice** is recognised from the **array shape** (a 4-D TWIX array = many slices), so a multi-slice Siemens scan prompts you for a slice index regardless of its name.
- **The separate GRAPPA ACS block:** native GRAPPA stores its calibration lines in a separate `refscan` measurement, not in the image data. The Equispaced recon **detects this by finding the ACS region empty** in the image k-space and then pulls the `refscan` ACS in automatically — again independent of the name (custom sequences fill their ACS lines, so they never trigger it).

Recommended names (keep the pair in the **same** `dat_files/` folder):

```
subject_ … /dat_files/
├── Siemens_meas_ … _t1_se_tra_fully_sampled.dat            ← SE, fully sampled (reference)
├── Siemens_meas_ … _t1_fl2d_tra_equispaced_R2_acs24.dat    ← GRE GRAPPA, R2
└── Siemens_meas_ … _t1_fl2d_tra_equispaced_R4_acs24.dat    ← GRE GRAPPA, R4
```

> **Note on the reference:** the fully-sampled Siemens reference is usually a **spin-echo** (`t1_se`) while the accelerated scan is a **gradient-echo** (`t1_fl2d`). The app still compares them but flags a *sequence mismatch* on the result, because SSIM/PSNR then reflect SE-vs-GRE contrast differences, not just the reconstruction error. For a clean metric, provide a fully-sampled scan of the **same** sequence. See §4c.

---

3. Launching the app

- **Standalone app:** open `SHIR.exe` (Windows) or `SHIR.app` (macOS) from its unzipped folder. First-time setup: `demands.md` / `demands_mac.md` .
- **From source (developers):** run `python app.py` from `AnalyzeMRIDataApp/` .

The window has three panels — **Folder** (left), **Results** (center, tabbed), **Parameters** (right) — with a toolbar on top and a **progress bar + log console** at the bottom.

Toolbar: **Open · Run Selected · Simulate · Stop · All Coils · Clear Tabs · PDF · About**

---

4. Step-by-step workflow

1. **Open** a scan folder (toolbar **Open** or the folder panel). Files appear in a colour-coded tree by type (green=Band, blue=Fully-Sampled, yellow=Equispaced, pink=Poisson).

2. **Select** what to process by ticking folders / types / individual files. For **Band** files, use the band-index checkboxes (0–15) at the bottom of the folder panel to choose which bands. **Select All / Clear All** for bulk.
3. **Set parameters** in the right panel (see §7), including the **Recon engine** and the per-type method.
4. **Run Selected.** Jobs run one after another in the background; watch the log console. **Stop** cancels after the current job.
5. **View results** — each finished recon opens as a new **tab** (see §8).
6. **Generate PDF** to export a report (see §9).

Multi-slice / TSE prompts

- If a scan has multiple slices and you did **not** pre-select slices (below), a dialog asks which single **slice index** to reconstruct (defaults to the middle slice).
- If an equispaced file has no matching JSON, the app asks how to build the PE **mask** (analytical vs. from the data).

4b. Reconstructing multiple slices

By default a multi-slice scan reconstructs **one** slice (the dialog above). To reconstruct several slices at once, **right-click the scan** → **"Select slices..."**. The scan expands into a checkbox per slice (`slice 0 … slice N-1`), all ticked; the scan's own checkbox is the **"All slices"** master (untick it, then tick just the slices you want). Each ticked slice is reconstructed into its **own result tab**, labelled `… – slice k/N` (and its own PDF page). Right-click → "Collapse slices (single)" returns to the one-slice behaviour.

Multi-slice applies to the Cartesian recons (Fully-Sampled, Equispaced, the Simulator, and Unknown/generic). **Band and Poisson-disc are single-slice** (non-Cartesian NUFFT), so no per-slice option is shown for them.

4c. How the reference is chosen (undersampled scans)

Every **Equispaced** and **Poisson-disc** undersampled scan is compared against a **fully-sampled reference** (reconstructed image + absolute difference + SSIM / NMSE / PSNR). By default the app finds the reference automatically, **within the same** `dat_files/` **folder**, right before the run:

1. It collects up to two reference candidates from that folder: - **Custom fully-sampled** — any file whose name contains `fully_sampled` . - **Siemens SE** — any spin-echo file (`t1_se_tra` / `_se_tra` / `t1_se_`) that is *not* itself an accelerated/other-type file (i.e. does not contain `grappa` , `fl2d` , `equispaced` , `poisson` , or `band`).
2. It then matches by the **kind** of the undersampled scan: - A **Siemens GRAPPA** scan (name contains `Siemens_` , `t1_fl2d_tra` , or `_fl2d_tra`) → prefers the **Siemens SE** reference, falling back to the custom fully-sampled. - A **custom** scan → prefers the **custom fully-sampled** reference, falling back to the Siemens SE. - **Poisson-disc** just takes the folder's fully-sampled scan (custom preferred, else the Siemens SE).
3. If **no** reference is found, the recon still runs — just **without metrics**.
4. The reference is RSS-combined and, if its matrix size differs, resized to match. If the reference's **sequence** (SE / GRE / TSE) or **matrix size** differs from the reconstructed scan, the result is tagged with a **"reference mismatch"** warning: the metrics then also reflect sequence/contrast/FOV differences, not reconstruction error alone.

Choosing the reference yourself: right-click an undersampled scan → **"Set reference"** and pick one of the fully-sampled scans listed (found across your loaded folders), or **"Browse for .dat..."** to choose any file; **"Auto**

(default)" restores the automatic choice. A manually-set reference is never overwritten by the automatic matcher, and it propagates to all of a scan's selected slices.

So, in practice: **keep the accelerated scan and its fully-sampled reference in the same `dat_files/` folder, and give the fully-sampled one a `fully_sampled` name** (matching sequence for the cleanest metric). The **batch report** (`batch_report.py`) uses a simpler rule — it groups files as *Siemens* (name starts with `Siemens`) vs *custom* and uses each group's `fully_sampled` file as that group's reference.

> **Unrecognised / overridden scans:** a scan that isn't detected as Equispaced (e.g. an unnamed Siemens `fl2d` file) runs through the generic fallback and gets an image but **no reference/metrics**. To get metrics, name it with `equispaced` (+ `_R<n>_acs<n>`) or right-click → **Reconstruct as** → **Equispaced**; reference matching then applies as above.

---

5. Reconstruction abilities (by type)

Fully-Sampled — *Combination mode*: - **RSS** — fast, parameter-free root-sum-of-squares. - **ESPIRiT + SENSE** — sensitivity-map coil combination (uses *calib width*).

Equispaced — *Method*: - **GRAPPA (pygrappa)** — k-space kernel; no BART needed. - **GRAPPA + SENSE** — GRAPPA fill + ESPIRiT coil combination. - **Pure SENSE** / **CS-SENSE** — ESPIRiT maps + SENSE (CS adds L1-wavelet). - **ESPIRiT**, **+L1**, **+L2**, **+TV**, **+L1-Wav** — ESPIRiT recon with the chosen regularizer. - *Mask*: **analytical** (from R/ACS) or **from data** (sampling pattern).

Band — always NUFFT gridding → ESPIRiT + SENSE (per selected band index).

Poisson-Disc — *Recon mode*: - **Zero-Fill (RSS)** — adjoint NUFFT + RSS, fast. - **CS PICS (wavelet)** — L1-wavelet compressed sensing. - Compared against a fully-sampled reference when one is available (see §4c); the recon intensity is rescaled to the reference before the difference/metrics are computed.

Unknown — generic fallback (*Unknown Scans* parameter group, *Recon mode*): - **Auto** *(default)* — if a trajectory JSON is found next to the `.dat` (`<scanname>.json` / `Trajectory.json` , or one picked via right-click), grid it with NUFFT + RSS; otherwise plain Cartesian IFFT + RSS. - **Cartesian IFFT + RSS** — assume Cartesian sampling. - **NUFFT (trajectory JSON)** — force the NUFFT path. - **NUFFT (assume radial)** — generate a uniform radial trajectory with `bart traj -r` (spokes × samples from the data shape). Assumes *uniform* radial; golden-angle / interleaved ordering will look streaky. - **CS (mask from k-space)** — for undersampled Cartesian data: extract the sampling mask with `bart pattern` and run ESPIRiT + PICS L1-wavelet. - Any mode falls back to Cartesian IFFT + RSS on failure, so an unknown scan never errors out. Without a trajectory, a genuinely non-Cartesian scan will look uninterpretable — the log warns when this happens.

You can also **run a specific known pipeline** on an unknown scan: right-click the file in the folder tree → **Reconstruct as** → *Fully-Sampled*, *Equispaced*, or *Poisson-Disc CS* (or back to *Generic fallback*). The chosen type is shown as `[as …]` next to the filename and uses that pipeline's parameters. (Poisson-Disc needs a trajectory JSON — assign one via **Choose trajectory JSON…** in the same menu.)

> **gGRAPPA** (a torch-based GRAPPA in the Simulator) is **not available** in the standalone builds (they exclude torch). Use GRAPPA (pygrappa) instead.

---

6. Reconstruction engine (BART vs sigpy)

The **Recon engine** dropdown (top of the parameters panel) controls how ESPIRiT / SENSE / CS / NUFFT methods are computed:

- **Auto** *(default)* — use **BART** if it's installed, otherwise fall back to the built-in **sigpy** engine.
- **Force BART** — always BART (exact numerics; needs BART — via WSL on Windows, natively on macOS/Linux).
- **Force sigpy** — always the built-in engine; **no BART/WSL needed**.

BART and sigpy give very similar but **not identical** results (different solvers). sigpy runs on CPU and is somewhat slower. GRAPPA and RSS don't use either engine and always work. The log prints which engine ran for each job.

---

7. Parameter reference

Group	Parameter	Applies to	Meaning
Common	Grid size	Band, Poisson	NUFFT oversampled grid (e.g. 512)
Common	Scan FOV (mm)	Band, Poisson	FOV used to auto-compute crop size
Common	Crop size (px)	Band, Poisson	Output image size; 0 = auto from FOV
Common	Traj margin (px)	Band, Poisson	Keeps trajectory inside the grid
Common	ESPIRiT calib width	all ESPIRiT/SENSE	Calibration region for sensitivity maps
Common	**Recon engine**	all BART/sigpy methods	Auto / Force BART / Force sigpy
Fully Sampled	Combination mode	Fully-Sampled	RSS or ESPIRiT + SENSE
Equispaced	Method	Equispaced	GRAPPA … ESPIRiT ±L1/L2/TV
Equispaced	Mask method	Equispaced	analytical or from-data pattern
Equispaced	ACS lines / R-factor	Equispaced	calibration lines / acceleration
Equispaced	Kernel size, λ GRAPPA	Equispaced (GRAPPA)	kernel (rows×cols), Tikhonov weight
Equispaced	λ CS/ESPIRiT, SENSE iters, ESPIRiT maps	Equispaced (SENSE/ESPIRiT)	regularization, iterations, map sets
Poisson	Recon mode	Poisson	Zero-Fill or CS PICS
Poisson	λ wavelet, Max iterations	Poisson (PICS) & Unknown (CS)	L1 weight, CG iterations
Unknown Scans	Recon mode	Unknown	Auto / Cartesian RSS / NUFFT (JSON) / NUFFT (radial) / CS (pattern)
Simulator	Mask type	Simulator	Equispaced or Variable Density
Simulator	R, ACS lines, Power decay	Simulator	acceleration, centre lines, VD PDF exponent
Simulator	Recon method	Simulator	None / GRAPPA / SENSE / CS-SENSE / ESPIRiT*
Output	Save directory	all	where PDF/.npy go (default `results/`)
Output	Generate PDF / Save .npy	all	auto-export after jobs finish

8. Viewing results

Each completed reconstruction is a **tab** in the center viewer: - **W/L sliders** — window/level (min%/max% of the data range). - ⟳ **Rotate 90°** — rotate every image in the tab 90° (click again to keep turning until the anatomy is oriented the way you want). Rotation is **separate per tab**, and the orientation you choose is also used in the **PDF report** for that scan. - **matplotlib toolbar** — zoom, pan, and save the current figure as PNG. - **All Coils** — open a grid of every per-coil image for the active tab. - **Clear Tabs** — remove all result tabs.

Tab titles follow `‹folder› – ‹type›[ ‹detail›]`, e.g. `SE_1mm – Band 0`, `SE_1mm – Equispaced GRAPPA`, `SE_1mm – Retro Sim [equispaced R=4.0×]`.

---

9. Outputs & file naming

All outputs go to the **Save directory** (default: the app's `results/` folder).

- **PDF report** — one per scan folder: `report_<ScanFolder>.pdf`. Generated when you click **Generate PDF** (or automatically if *Generate PDF* is ticked). Includes the images, k-space, and (for the simulator) RMSE/PSNR panels.
- **NumPy arrays** — if *Save images (.npy)* is ticked, each result's combined image is saved as `<tab-title>.npy` (spaces→`_`, `–`→`-`), e.g. `SE_1mm_-_Band_0.npy`.
- **Individual figures** — save any single view as PNG via the matplotlib toolbar's save button.

---

10. The Retrospective Undersampling Simulator

Simulate undersampling on an already-acquired **Fully-Sampled** scan — no re-scanning: 1. Tick one or more **Fully-Sampled** files. 2. Configure the **Undersampling Simulator** group (mask type, R, ACS, decay, recon method). 3. Click **Simulate** in the toolbar.

It builds a 1-D PE mask (**Equispaced** or **Variable Density**), applies it to the fully-sampled k-space, and optionally reconstructs — producing a 3×3 result panel: reference vs zero-filled vs recon, the difference image, the mask, the k-spaces, and **RMSE/PSNR** vs the fully-sampled reference.

---

11. Tips & troubleshooting

- **A file shows as "Unknown":** its name doesn't match the rules in §2. It's still reconstructed with the generic fallback (Cartesian RSS by default). For a proper recon, rename it to contain `fully_sampled`, `equispaced`, `poisson`, or `band_se_N` / `band_tse_N`, or — for a non-Cartesian scan — add a `<scanname>.json` / `Trajectory.json` trajectory and pick a NUFFT mode in the **Unknown Scans** panel.
- **Band/Poisson do nothing:** the trajectory JSON is missing or misplaced — put `Bands.json` / `PoissonDisc.json` / `Undersampled<R>equispaced.json` in the scan-folder root.
- **"could not launch BART" (only in Force BART):** switch the engine to **Auto** or **Force sigpy**, or install BART (see the demands docs).
- **Recon seems slow:** you're likely on the sigpy engine (CPU) — install BART and use Auto/Force BART, or lower the iteration count.
- **Nothing but errors on a windowed build:** check the log file the app writes — `%TEMP%\SHIR.log` (Windows) or `$TMPDIR/SHIR.log` (macOS).
- **Equispaced R mismatch:** the R in the `.dat` name (`_R4`) must match an `Undersampled4equispaced.json`, and the **R-factor** parameter should agree.